01 — The Maths Behind Attention

Three questions the maths answers: which parts of a matrix matter, why training slows down, and what a loss function quietly assumes.

Reading time: about 45 minutes. You need: matrix multiplication, eigenvalues, the chain rule, and basic probability (expectations and logarithms). Module 00 helps but is not required. You do not need: any optimisation or statistics course.

Three puzzles

Here are three facts about modern models that look strange at first sight.

Puzzle 1: a large matrix that is secretly small. An attention head in a model with 64-number token vectors uses two matrices of size 64 × 8. Multiply them together and you get a 64 × 64 matrix, with 4,096 entries. Yet its rank is 8, not 64. We checked: `torch.linalg.matrix_rank` returns 8.

Puzzle 2: a simple problem that takes forever. Take the easiest function there is to minimise: a smooth bowl. Make it 100 times steeper in one direction than in another. Gradient descent, with the best possible fixed step size, now needs 115 steps to gain each extra digit of accuracy. Make it 10,000 times steeper, and it needs 11,513 steps per digit.

Puzzle 3: a choice nobody seems to make. Every classifier turns scores into probabilities with softmax. Why the exponential? Dividing each score by the total would also give numbers that add up to one.

Each puzzle has a clean explanation, using maths you already know. This article gives those explanations, and uses them to understand the attention function from module 00.

What you will learn

By the end of this article, you will be able to:

- use the singular value decomposition to find the best low-rank approximation of a matrix, and predict its error;
- explain what an attention head can and cannot express;
- relate the shape of a loss surface to how fast gradient descent converges;
- derive the gradient of attention by hand;
- connect loss functions to probability: cross-entropy as maximum likelihood, softmax as maximum entropy, and L2 regularisation as a prior.

Notation used across the course, including the several kinds of multiplication, is collected in GLOSSARY.md at the top of the repository. Look there whenever a symbol is unfamiliar.

The article has three parts. Part 1 is linear algebra (sections 1 to 3). Part 2 is calculus and optimisation (sections 4 to 6). Part 3 is probability (sections 7 to 9). A recap and the exercise briefing follow at the end.

Part 1: Linear algebra

1. Every matrix rotates, stretches, then rotates again

Let us start with a small matrix and ask what it does to space:

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

Apply A to every point on a circle of radius 1, and you get an ellipse. The **singular value decomposition**, or SVD, describes exactly how. It writes any matrix as three simpler steps:

$$A = U\Sigma V^\top$$

Read it from right to left, the order in which it acts on a vector. $V^\top$ rotates. Σ is diagonal, so it stretches along the coordinate axes. U rotates again. The stretch factors on the diagonal of Σ are the **singular values**, always listed from largest to smallest.

For our matrix, PyTorch gives singular values of 6.708 and 2.236. So the ellipse has a long axis of length 6.708 and a short axis of length 2.236:

```
unit circle          after A: an ellipse
   .--.              long axis:  6.708
  /    \      A      short axis: 2.236
  \    /    ---->
   '--'
```

The number of non-zero singular values is the **rank** of the matrix. It counts how many independent directions the matrix can stretch at all.

Keeping only the largest stretch

Suppose we must describe A with a rank-1 matrix, using only one direction. The natural choice is to keep the largest singular value and drop the other.

Dropping one is mechanical, once you write the SVD as a sum with one term per singular value:

$$A = \sigma_1 u_1 v_1^\top + \sigma_2 u_2 v_2^\top$$

Here u_i is column i of U and v_i is column i of V. Each term is a column times a row, which is an outer product: a full matrix built from two vectors, and the simplest kind of matrix there is. Keeping only the first term leaves $A_1 = \sigma_1 u_1 v_1^\top$.

For our matrix, PyTorch returns these vectors:

$$\sigma_1 = 6.708, \qquad u_1 = \begin{bmatrix} -0.316 \\ -0.949 \end{bmatrix}, \qquad v_1 = \begin{bmatrix} -0.707 \\ -0.707 \end{bmatrix}$$

Multiply them out. The top-left entry is $6.708 \times (-0.316) \times (-0.707) = 1.5$, and the other three follow the same way:

$$A_1 = 6.708 \begin{bmatrix} -0.316 \\ -0.949 \end{bmatrix} \begin{bmatrix} -0.707 & -0.707 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 4.5 & 4.5 \end{bmatrix}$$

Two details are worth noting. The two columns of A_1 are identical, which is what rank 1 looks like: every output lies along the single direction u_1. And the signs of u_1 and v_1 are arbitrary, because flipping both leaves the product unchanged, so your library may return the opposite pair.

The squared error of this approximation, summed over all entries, is exactly 5.0. That is no coincidence: $2.236^2 = 5$. The error equals the square of the singular value we threw away.

This holds for every matrix, and it is the best you can do. The **Eckart–Young theorem** states it precisely:

$$\min_{\text{rank}(B)\le r} \|A - B\|_F^2 \;=\; \sum_{i>r} \sigma_i^2$$

In words: the best rank-r approximation keeps the r largest singular values, and its squared error is the sum of the squares of all the others. No other rank-r matrix comes closer.

Why this matters

Many trained weight matrices have singular values that shrink quickly. For those matrices, a low-rank version loses very little, which is why compression and principal component analysis work.

LoRA (low-rank adaptation) uses the same idea for fine-tuning. A 4,096 × 4,096 weight matrix has 16.8 million entries. LoRA learns an update $\Delta W = BA$ with rank 8 instead, which needs only 65,536 numbers: 256 times fewer. The bet is that the *change* a new task needs is low-rank, even when the original matrix is not.

---

2. An attention head compares tokens through a narrow window

Now we can solve puzzle 1. Recall from module 00 that attention compares each query with each key. Let us write out where queries and keys come from.

Each token starts as a vector x of length 64. The head projects it down to length 8 twice: once to make a query, and once to make a key.

Symbol	Meaning	Shape
x_i	token i, as a row vector	1×64
W_Q	the query projection	64×8
W_K	the key projection	64×8
$q_i = x_i W_Q$	query of token i	1×8
$k_j = x_j W_K$	key of token j	1×8

The score between token i and token j is the dot product of q_i and k_j. Expand it, and the two projections meet in the middle:

$$q_i k_j^\top = x_i W_Q \, W_K^\top x_j^\top = x_i \, (W_Q W_K^\top) \, x_j^\top$$

The middle matrix $W_Q W_K^\top$ is 64×64. But the rank of a product can never exceed the smallest dimension it passes through, and here that is 8. So its rank is at most 8.

What does that mean in practice? The head never compares two full 64-number vectors. It first squeezes each one down to 8 numbers, then compares those. It cannot express an arbitrary rule about which tokens should attend to which.

This is a design choice, not an accident. **Multi-head attention** runs several heads side by side, each with its own 8-dimensional window. For the same number of parameters, you get several different small comparisons instead of one big one.

The two projections are not unique

There is a second surprise hiding in the formula. Pick any invertible 8 × 8 matrix R, and change the projections like this:

$$W_Q \to W_Q R, \qquad W_K \to W_K (R^{-1})^\top$$

The product in the middle does not change:

$$W_Q R \left(W_K (R^{-1})^\top\right)^\top = W_Q R\, R^{-1} W_K^\top = W_Q W_K^\top$$

So every attention score stays exactly the same. Infinitely many different pairs (W_Q, W_K) describe the same head. Studying W_Q on its own is therefore meaningless; only the product carries information. Interpretability research calls this product the *QK circuit*, and studies it instead of the individual matrices.

> **In the interview:** "a head's attention pattern depends only on $W_Q W_K^\top$, which has rank at most d_{head}" is a compact, strong answer to "what can one attention head represent?".

---

3. Eigenvalues decide how fast gradient descent converges

Now puzzle 2. Let us build the slow problem and watch where the time goes.

Take the bowl $f(x) = \frac{1}{2} x^\top A x$ with $A = \text{diag}(1, 100)$. Along the first coordinate the bowl is gentle; along the second it is 100 times steeper. The minimum is at $x = 0$.

Gradient descent repeats one update, with a step size η:

$$x \leftarrow x - \eta\, A x$$

Because A is diagonal, each coordinate evolves separately. Each step multiplies coordinate i by $(1 - \eta\lambda_i)$, where λ_i is the steepness in that direction.

The step size is caught between two directions

The steep direction limits the step size. If η is too large, coordinate 2 overshoots the bottom and bounces further away each time. The flat direction then pays for that small step, and makes painfully slow progress.

The best fixed step balances the two: $\eta = 2/(\lambda_{\min} + \lambda_{\max}) = 2/101$. With that choice, both coordinates shrink by the same factor on every step:

$$\left|1 - \tfrac{2}{101} \cdot 1\right| \;=\; \left|1 - \tfrac{2}{101} \cdot 100\right| \;=\; \tfrac{99}{101} \;\approx\; 0.9802$$

To gain one digit of accuracy, the error must shrink by a factor of 10. Solving $0.9802^n = 0.1$ gives $n \approx 115$ steps. A direct simulation agrees: shrinking the error a millionfold takes 691 steps, against a prediction of 690.8.

The general rule

The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ is called the **condition number**.

It is worth being careful about what belongs to what. Each λ_i belongs to one direction: here $\lambda = 1$ along the first coordinate and $\lambda = 100$ along the second. The condition number belongs to the surface as a whole, because it compares the steepest direction with the flattest one. So κ is not a property of any single dimension. It is the gap between the two extremes, and that gap is the problem, because one step size has to serve both ends of it.

Our example is diagonal, so its directions are the coordinate axes and its λ_i sit on the diagonal. In general the directions are the eigenvectors of A, which can point anywhere, and the λ_i are the matching eigenvalues. The reasoning does not change: κ still compares the steepest eigenvector direction with the flattest one.

For any such bowl, gradient descent with the best fixed step shrinks the error by a factor of $(\kappa - 1)/(\kappa + 1)$ per step. The cost grows roughly in proportion to κ:

Condition number κ	Shrink factor per step	Steps per digit of accuracy
10	0.8182	11.5
100	0.9802	115
10,000	0.9998	11,513

This explains a common sight in training. A loss curve that flattens out is often not stuck in a bad minimum. It is crawling along a long, narrow valley, where some directions are far steeper than others.

Many standard training techniques exist to reduce κ. Normalisation layers, careful weight initialisation and adaptive optimisers such as Adam all do it in different ways. Section 6 returns to Adam.

One more link back to section 1: the largest singular value of a matrix is the most it can stretch any vector. It bounds how much a layer can amplify a signal, which is why it matters for stable training.

Part 2: Calculus and optimisation

4. Backpropagation multiplies Jacobians, starting from the loss

Module 00 showed that `.backward()` computes vector–Jacobian products. Here is why that works for a whole network.

A network is a chain of functions: $f = f_L \circ \cdots \circ f_2 \circ f_1$. The chain rule says its Jacobian is the product of the individual Jacobians:

$$J_f = J_L \cdots J_2 J_1$$

Matrix multiplication can be done in any grouping, and the grouping decides the cost. **Reverse mode**, which is what backpropagation uses, starts from the loss end.

A bar over a symbol means "the gradient of the loss with respect to this symbol". So $\bar{y}$ holds one number for each entry of y, saying how much the loss changes when that entry changes. A barred quantity therefore always has the same shape as the quantity under the bar.

Reverse mode multiplies the row vector $\bar{y}^\top$ by J_L, then the result by J_{L-1}, and so on. Every product is a vector times a matrix, so every product is cheap.

Forward mode starts from the input end instead. It would need a separate pass for every input. With billions of parameters as inputs and one loss as output, reverse mode is the only practical choice.

There is a price. Most Jacobians need the values computed during the forward pass, so those values must be kept in memory until backward uses them. This is why training needs far more memory than running a model, and module 04 shows how to trade some recomputation for less memory.

---

5. Deriving the gradient of attention

This derivation looks intimidating when written all at once. We will take it one step at a time, and each step uses only two small rules.

The forward pass, in three steps

Step	Computation	Shape
1. scores	$S = QK^\top/\sqrt{d}$	$T \times T$
2. weights	$P = \text{softmax}(S)$, row by row	$T \times T$
3. output	$O = PV$	$T \times d_v$

A warning about letters. The V in this section is attention's value matrix, not the V of the SVD in section 1. Both uses are standard, and this course keeps both rather than inventing a private notation.

The two rules we need

For backward, we use the bar notation from section 4, now applied to matrices. We write $\bar{O}$ for the gradient of the loss with respect to O, meaning the matrix whose (i, j) entry is $\partial\mathcal{L}/\partial O_{ij}$:

$$\bar{O}_{ij} = \frac{\partial \mathcal{L}}{\partial O_{ij}} \qquad \text{so} \qquad \bar{O} \text{ has the same shape as } O$$

The curly $\mathcal{L}$ is the loss. Plain L already means the number of layers, in section 4.

One number per entry, each saying how the loss responds to that entry. This is the same object module 00 called the cotangent, and the same thing PyTorch stores in a tensor's `.grad`. The bar is only a short way to write it. The same notation applies to every other matrix below, so $\bar{Q}$, $\bar{K}$, $\bar{V}$, $\bar{S}$ and $\bar{P}$ each have the shape of the matrix they decorate.

The matmul rule. If $C = AB$, then

$$\bar{A} = \bar{C}B^\top, \qquad \bar{B} = A^\top\bar{C}$$

You can check it with shapes. If A is $n \times k$ and B is $k \times m$, then $\bar{C}$ is $n \times m$, and $\bar{C}B^\top$ is $n \times k$, the shape of A.

The softmax rule. For one row with weights p and incoming gradient $\bar{p}$:

$$\bar{s} = p \odot \left(\bar{p} - \langle \bar{p}, p \rangle\right)$$

Here $\odot$ multiplies entry by entry, and $\langle \bar{p}, p \rangle$ is a dot product: a single number. You derived this rule in exercise 00, step 4.

Walking backwards

Now go through the three forward steps in reverse order.

Step 3 backwards. $O = PV$ is a matrix product, so the matmul rule gives:

$$\bar{V} = P^\top \bar{O}, \qquad \bar{P} = \bar{O} V^\top$$

Step 2 backwards. Apply the softmax rule to each row. Let $r_i = \langle \bar{p}_i, p_i \rangle$ be the dot product for row i. Then row i of $\bar{S}$ is:

$$\bar{s}_i = p_i \odot (\bar{p}_i - r_i)$$

Step 1 backwards. $S = QK^\top/\sqrt{d}$ is a matrix product with a constant factor, so:

$$\bar{Q} = \frac{1}{\sqrt{d}} \bar{S} K, \qquad \bar{K} = \frac{1}{\sqrt{d}} \bar{S}^\top Q$$

That is the whole derivation. Checked against PyTorch's autograd on a small example, the largest difference in any entry was 5.6×10^{-17}, which is rounding error.

What the result tells us

The formulas are worth reading closely, because each one says something useful.

Backward costs about twice the forward pass. It performs the same kinds of work: matrix products and a softmax-like step, roughly twice as many of them.

Gradient flows only where attention was paid. In step 2, $\bar{S}$ is multiplied entry by entry by P. If the softmax puts almost all its weight on one token, most entries of P are near zero, and little gradient reaches Q and K. On a small test, multiplying Q by 20 made the weights much more concentrated: their entropy fell from 1.46 to 0.19. The size of $\bar{Q}$ fell from 1.71 to 0.92. This is the gradient-side reason for the $\sqrt{d}$ scaling from module 00.

You never need to store P to get r_i. Row i of $\bar{P}$ is $\bar{o}_i V^\top$, so:

$$r_i = \bar{o}_i V^\top p_i^\top = \bar{o}_i \, (p_i V)^\top = \bar{o}_i \, o_i^\top$$

In words, r_i is the dot product of the output with its own gradient. Both are small vectors, so nothing of size $T \times T$ is needed. This one-line identity is what lets FlashAttention, in module 07, compute the backward pass with far less memory.

In the interview: being able to write $\bar{V}$, $\bar{P}$, $\bar{S}$, $\bar{Q}$ and $\bar{K}$ in order, with shapes, is a common whiteboard task. Say the two rules first; the rest follows.

---

6. Why training uses Adam, warm-up and clipping

Section 3 used a perfect bowl. Real loss surfaces are not bowls, and that changes which tools we need.

A function is **convex** if every local minimum is also the global minimum. Neural network losses are not convex. Swapping two hidden units, for instance, gives a different set of weights that computes exactly the same function, so there are always many equally good minima.

In practice, though, bad local minima are rarely the problem. The obstacles are badly conditioned regions, like the valley in section 3, and **saddle points**: places that curve upwards in some directions and downwards in others. In high dimensions, almost every flat point is a saddle. Gradient noise from small batches helps training move past them.

Three standard tools address what remains:

- **Adam** keeps a running average of each gradient coordinate and of its square. It divides each step by the square root of that second average, so every coordinate takes a similar-sized step. This acts like a diagonal fix for the condition number. For transformers it usually works much better than plain gradient descent.
- **AdamW** fixes a subtle problem in Adam. In plain Adam, an L2 penalty passes through the same division, so it no longer behaves like true weight decay. AdamW applies the decay separately.
- **Warm-up** starts with a small learning rate. Early on, Adam's averages are based on only a few gradients and are unreliable, so large steps would be risky.
- **Gradient clipping** caps the size of the gradient. One unusual batch then cannot throw the weights far away.

Part 3: Probability

7. Minimising cross-entropy is maximum-likelihood estimation

Let us start with something concrete. You roll a loaded four-sided die 100 times and see these counts:

Face	1	2	3	4
Count	12	30	5	53

Your model assigns a probability to each face. Which probabilities fit best?

Maximum-likelihood estimation picks the probabilities that make the observed rolls as likely as possible. Multiplying many small probabilities is awkward, so we take logarithms and flip the sign. That gives the quantity we minimise, the average **negative log-likelihood**:

$$\text{NLL} = -\frac{1}{N}\sum_{i=1}^{N} \log p(x_i)$$

Read it one piece at a time. x_i is observation i, here one roll of the die. $p(x_i)$ is the probability your model gave to the face that actually came up. The logarithm turns the product of all those probabilities into a sum, which is easier to work with and does not underflow. Every $\log p$ is negative, because probabilities are below 1, so the minus sign makes the total positive. The $1/N$ averages over the observations, which keeps the number comparable between datasets of different sizes.

The meaning is average surprise. A model that gave probability 1 to everything that happened scores 0, since $\log 1 = 0$. The less probable the model thought the actual data was, the larger the score, and a model that gave probability 0 to something that happened scores infinity.

Grouping the 100 rolls by face shows where the number comes from. Take the model $p = (0.12, 0.30, 0.05, 0.53)$:

Face	Count	p	$-\log p$	Share of the average
1	12	0.12	2.120	0.254
2	30	0.30	1.204	0.361
3	5	0.05	2.996	0.150
4	53	0.53	0.635	0.337
total	100			1.102

Face 3 is the most surprising single roll, at 2.996, but it is rare, so it contributes least to the average. Face 4 is the least surprising and contributes most, because it happens so often. The average of 1.102 is the negative log-likelihood of this model on this data.

For the die, the best answer is the observed frequencies: 0.12, 0.30, 0.05 and 0.53, which are the very numbers used above. Any other model scores higher. A model that calls all four faces equally likely scores $\ln 4 = 1.386$. Exercise step 5 confirms the best answer by gradient descent.

One convention: logarithms in this course are natural logarithms, so these numbers are in nats. Using $\log_2$ instead gives the same quantity in bits, which is 1.44 times larger.

Three names for one quantity

The negative log-likelihood splits into two parts:

$$\text{NLL} = H(\hat{p}) + D_{\text{KL}}(\hat{p} \,\|\, p)$$

Here $\hat{p}$ is the distribution of the data, and p is the model.

- $H(\hat{p})$ is the **entropy** of the data: how unpredictable the rolls are. It does not depend on the model at all.
- $D_{\text{KL}}(\hat{p}\|p)$ is the **KL divergence**: how far the model is from the data. It is never negative, and it is zero only when the two match.

Since the entropy is fixed, minimising the negative log-likelihood is the same as minimising the KL divergence. And this quantity is exactly the **cross-entropy** loss that classifiers use. So three names describe one thing.

The die makes this concrete. The data's entropy is 1.102. A model that says every face is equally likely has a negative log-likelihood of $\ln 4 = 1.386$, so its KL divergence is 0.284. The best model reaches a KL divergence of 0, and a negative log-likelihood of 1.102.

Perplexity

Language models often report **perplexity**, which is e^{NLL}. It measures uncertainty as a number of equally likely choices. The best die model has a perplexity of 3.01: it is as unsure as someone choosing between about three equally likely faces. A language model with a perplexity of 20 is as unsure as someone choosing among 20 equally likely next words.

The direction of the divergence matters

KL divergence is not symmetric. Maximum likelihood minimises $D_{\mathrm{KL}}(\text{data} \,\|\, \text{model})$, which punishes the model severely for giving near-zero probability to anything that appears in the data. So models trained this way tend to spread probability widely and hedge.

The reverse direction, $D_{\mathrm{KL}}(\text{model} \,\|\, \text{data})$, behaves the opposite way. It lets the model ignore rare cases and concentrate on the most common one. It appears in the fine-tuning methods of module 08.

---

8. Softmax is the least-committal choice

Now puzzle 3: why the exponential?

A worked example

Take five scores: $z = (-2, -0.5, 0, 1, 3)$. Suppose the only thing we require of a probability distribution is that the average score under it equals 1.0. Many distributions satisfy that. Which one should we choose?

A principled answer is: choose the one that assumes the least. Entropy measures how spread out a distribution is, so we pick the distribution with the highest entropy that still meets the requirement. This is the **principle of maximum entropy**.

Solving that problem numerically gives:

$$p = (0.105,\ 0.151,\ 0.171,\ 0.218,\ 0.356)$$

Its average score is exactly 1.0, and its entropy is 1.523. For comparison, the uniform distribution has a higher entropy of 1.609, but its average score is 0.3, so it fails the requirement.

Now the surprising part: those five numbers are exactly $\mathrm{softmax}(\beta z)$ with $\beta = 0.2446$. The next part explains why a softmax has to appear. The number 0.2446 itself has a simpler job. It is the dial that makes the average come out at 1.0, and nothing more. Turning it up leans the distribution towards the high scores, which raises the average:

β	average score under $\mathrm{softmax}(\beta z)$
0	0.300, the uniform distribution
0.1	0.582
0.2	0.871
0.2446	1.000, the requirement
0.3	1.158
0.5	1.688

The average rises steadily with β, so exactly one value hits 1.0. A dozen steps of bisection find it to four digits. The digits are not worth memorising: they are the answer to a one-dimensional search, and a different requirement would give a different β.

Why it is always softmax

The result holds in general. We want to maximise entropy subject to two constraints:

$$\text{maximise } -\sum_i p_i \log p_i \quad \text{subject to} \quad \sum_i p_i z_i = c \ \text{ and } \ \sum_i p_i = 1$$

Add a Lagrange multiplier for each constraint, β and ν, and set the derivative with respect to each p_i to zero:

$$-\log p_i - 1 + \beta z_i + \nu = 0$$

Rearrange to isolate p_i:

$$p_i = e^{\nu - 1}\, e^{\beta z_i} \ \propto \ e^{\beta z_i}$$

That is a softmax. The multiplier β is tuned until the average score comes out right. So softmax is not an arbitrary choice: it is the distribution that adds no assumptions beyond what the scores say.

Temperature

In practice we usually write β as $1/T$ and call T the **temperature**. Here is its effect on the scores $(2, 1, 0)$:

Temperature T	Resulting probabilities
0.25	0.982, 0.018, 0.000
1	0.665, 0.245, 0.090
4	0.419, 0.327, 0.254
100	0.337, 0.333, 0.330

Low temperatures put almost everything on the largest score. High temperatures approach the uniform distribution. You will meet temperature again when sampling text in module 05.

A second reason: clean gradients

Softmax also pairs perfectly with cross-entropy. The gradient of the loss with respect to the scores is the prediction minus the target:

$$\frac{\partial \,\text{loss}}{\partial z} = p - y$$

For example, take scores $(2, 1, 0.1)$ with class 0 correct. The probabilities are $(0.659, 0.242, 0.099)$, and the target is $(1, 0, 0)$. PyTorch reports a gradient of $(-0.341, 0.242, 0.099)$: exactly their difference.

This clean form matters. Compare a sigmoid output trained with squared error. There, the gradient keeps an extra factor, the sigmoid's slope, which is nearly zero when the output is saturated. A saturated unit then stops learning. With cross-entropy, that factor cancels out.

---

9. L2 regularisation is a Gaussian prior

First, the names. L1 and L2 are the two usual ways to measure the size of a weight vector:

$$\|w\|_1 = \sum_i |w_i| \qquad \text{and} \qquad \|w\|_2 = \sqrt{\textstyle\sum_i w_i^2}$$

The first adds up distances along the axes, the second is ordinary straight-line length. An L2 penalty adds $\lambda\|w\|_2^2$ to the loss, usually written $\lambda\|w\|^2$ once the subscript is clear, and an L1 penalty adds $\lambda\|w\|_1$. Statistics calls the two ridge and lasso regression; in deep learning the L2 version is close to what optimisers call weight decay.

The last idea connects regularisation to probability. It uses **Bayes' rule**: the probability of the weights given the data is proportional to the likelihood times the **prior**. The prior describes what we believed about the weights before seeing any data.

$$p(w \mid \text{data}) \;\propto\; p(\text{data} \mid w)\; p(w)$$

Maximising this is called **MAP estimation**. Taking negative logarithms turns it into a minimisation: the negative log-likelihood plus the negative log of the prior.

A worked example: linear regression

Assume the data follows $y = Xw +$ noise, where the noise is Gaussian with variance σ^2. Assume also the prior belief that each weight is Gaussian around zero, with variance τ^2. The objective to minimise becomes:

$$\frac{\|y - Xw\|^2}{2\sigma^2} + \frac{\|w\|^2}{2\tau^2}$$

The second term is an L2 penalty. So L2 regularisation is exactly a Gaussian prior on the weights.

The constant in front of the penalty depends on how you write the loss, which often causes confusion:

- Written as above, the penalty's coefficient is $1/(2\tau^2)$.
- Multiply everything by $2\sigma^2$, and the loss becomes plain squared error, $\|y - Xw\|^2 + \lambda\|w\|^2$, with $\lambda = \sigma^2/\tau^2$.

For example, with $\sigma^2 = 0.25$ and $\tau^2 = 2$, the squared-error form uses $\lambda = 0.125$. The exercise uses this form. The formula also matches intuition. A stronger belief that weights are small means a smaller τ^2, and so a larger penalty. Noisier data means a larger σ^2, so we trust the data less and regularise more.

A Laplace prior, which has a sharp peak at zero, gives an L1 penalty instead. That is where L1's tendency to set weights exactly to zero comes from. The reason is visible in the gradients. The gradient of λw^2 is $2\lambda w$, which fades as w approaches zero, so L2 shrinks a useless weight for ever without finishing. The gradient of $\lambda|w|$ is $\pm\lambda$, a push of constant size however small w gets, so weights the data does not support reach exactly zero and stay there.

What maximum likelihood does not give you

Large networks trained by maximum likelihood are often overconfident. When such a model says "90% sure", it is right less than 90% of the time. A cheap fix is **temperature scaling**: fit a single temperature on held-out data, as in section 8.

It also helps to separate two kinds of uncertainty. **Aleatoric** uncertainty is noise in the data itself, and more data cannot remove it. **Epistemic** uncertainty comes from not knowing the right weights, and more data does reduce it.

Recap

- The SVD writes any matrix as rotate, stretch, rotate. Keeping the largest singular values gives the best low-rank approximation, with a predictable error.
- An attention head compares tokens through a window of rank d_{head}, and only the product $W_Q W_K^{\mathsf{T}}$ matters.
- Gradient descent slows down in proportion to the condition number. Many training techniques exist to reduce it.
- The attention gradient follows from two rules: the matmul rule and the softmax rule. Gradient flows only where attention was paid.
- Minimising cross-entropy is maximum-likelihood estimation. Softmax is the maximum-entropy distribution, and its gradient with cross-entropy is prediction minus target.
- L2 regularisation is a Gaussian prior on the weights, with $\lambda = \sigma^2/\tau^2$ in the squared-error form.

The exercise: turn each claim into a measurement

What you will build. Seven small experiments, each of which checks one result from this article with real numbers. You will confirm the Eckart–Young error formula, watch gradient descent slow down exactly as predicted, recover softmax from maximum entropy, and more. The centrepiece is step 6: your own hand-derived attention gradient, checked against PyTorch's autograd.

Time and setup. Allow three to four hours. The setup is the same as for module 00; if you have already created the `.venv` environment, there is nothing new to install.

How to run it. Open `01-math-foundations/exercise.ipynb` and run the cells in order, or run `python exercise.py` from inside the module folder. Everything works in double precision (`float64`), because several checks are too precise for standard 32-bit numbers.

How the checks work. They work as in module 00. Replace each `raise NotImplementedError` with your code, then run the check cell below it. Some cells also print a table. Once step 3 is correct, for example, it prints this:

```
     kappa      empirical         theory    steps per digit
         2     0.33333333     0.33333333                2.1
[ok]   κ=2 matches theory
        10     0.81818182     0.81818182               11.5
[ok]   κ=10 matches theory
       100     0.98019802     0.98019802              115.1
[ok]   κ=100 matches theory
```

The "empirical" column is what your gradient descent measured; "theory" is $(\kappa - 1)/(\kappa + 1)$.

Step map. This table links each step to the part of the article it checks.

Step	You write	The result it checks	Section
1	`truncated_svd`, `predicted_error`	the Eckart–Young error formula	1
2	`qk_scores`, `gauge_transform`	a head has rank at most 8; W_Q and W_K are not unique	2
3	`gd_on_quadratic`, `gd_preconditioned`	the $(\kappa - 1)/(\kappa + 1)$ rate, and how rescaling helps	3, 6
4	`beta_for_mean`, `max_entropy_distribution`, `feasible_directions`	softmax is the maximum-entropy distribution	8
5	`negative_log_likelihood`, `forward_kl`	NLL equals KL plus entropy; the gradient is $p - y$	7, 8
6	`attention_forward`, `attention_backward`	the hand-derived gradient, saturation, and the r_i identity	5
7	`ridge`, `posterior_mean`	L2 regularisation equals a Gaussian prior, with $\lambda = \sigma^2/\tau^2$	9

A few steps end with a short written question, labelled 2c, 3b, 4c and 7b. Answer them in a sentence or two. `solution.py` includes model answers as comments.

When you are done. Every check passes. In step 6, your gradients match autograd to within 10^{-10}, and they also pass `torch.autograd.gradcheck`, which compares them with finite differences. At that point you have derived and verified the backward pass of attention yourself.

If you get stuck.

- Work with 2 × 2 or 3 × 3 matrices first, where you can check results by hand.
- For step 6, write the shape next to every line and compare with the table in section 5.
- Compare a gradient with a finite difference: nudge one input by 10^{-6} and see how the output changes.
- If you are still stuck, `solution.py` has one correct answer to every step, with comments explaining the choices.

Want more? The notebook ends with three stretch goals. You can measure the singular values of a real model's attention heads, or check how much attention can amplify its input. You can also compare the Hessian, which is the matrix of second derivatives, with the Fisher information matrix, a common approximation to it.

Going deeper

Start here

- 3Blue1Brown, *Essence of Linear Algebra* — https://www.3blue1brown.com/topics/linear-algebra. A short video series that makes matrices, rank and eigenvectors visual. Watch it first if part 1 felt fast.

Linear algebra

- Strang (2019), *Linear Algebra and Learning from Data*, Wellesley–Cambridge Press. Written for exactly this audience. Chapter I.9 covers Eckart–Young, and part III covers low-rank structure in learning.
- Trefethen & Bau (1997), *Numerical Linear Algebra*, SIAM. Lectures 4 and 5 give the clearest short treatment of the SVD; lecture 12 covers conditioning.
- Eckart & Young (1936), *The approximation of one matrix by another of lower rank*, Psychometrika 1(3). The original theorem from section 1, in about one page of mathematics.
- Hu et al. (2021), *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685. The fine-tuning method from section 1.
- Elhage et al. (2021), *A Mathematical Framework for Transformer Circuits*, Anthropic — https://transformer-circuits.pub/2021/framework/. The QK-circuit idea from section 2, developed fully. The best next step for understanding what attention heads do.

Optimisation and differentiation

- Boyd & Vandenberghe (2004), *Convex Optimization*, Cambridge University Press — https://web.stanford.edu/~boyd/cvxbook/. Free to download. Section 9.3 derives the convergence rate from section 3.
- Nocedal & Wright (2006), *Numerical Optimization*, 2nd ed., Springer. Chapters 3 and 6, for a deeper treatment of step sizes and preconditioning.
- Kingma & Ba (2015), *Adam: A Method for Stochastic Optimization*, ICLR. arXiv:1412.6980. The original Adam paper.
- Loshchilov & Hutter (2019), *Decoupled Weight Decay Regularization*, ICLR. arXiv:1711.05101. Section 2 explains precisely why Adam's L2 penalty is not weight decay.
- Dauphin et al. (2014), *Identifying and attacking the saddle point problem in high-dimensional non-convex optimization*, NeurIPS. arXiv:1406.2572. The evidence behind the claim about saddle points in section 6.

Probability and information theory

- MacKay (2003), *Information Theory, Inference, and Learning Algorithms*, Cambridge University Press. Free from the author's website. The most enjoyable route to entropy and Bayesian inference; see chapters 22, 23 and 28.
- Cover & Thomas (2006), *Elements of Information Theory*, 2nd ed., Wiley. Chapter 2 covers entropy and KL divergence; chapter 12 covers maximum entropy.
- Bishop (2006), *Pattern Recognition and Machine Learning*, Springer. Section 4.3 explains why softmax and cross-entropy pair so well; section 3.3 covers MAP estimation and regularisation.
- Jaynes (1957), *Information Theory and Statistical Mechanics*, Physical Review 106(4). The original argument for the principle of maximum entropy.
- Guo et al. (2017), *On Calibration of Modern Neural Networks*, ICML. arXiv:1706.04599. The overconfidence problem from section 9, and temperature scaling as a fix.

Next: module 02 turns these derivations into code, and builds a complete transformer block without autograd.